

Module 9 Lab Exercise 2 - Backend Database via MongoDB

Module 9 - Lab Exercise 2 (Page 29)

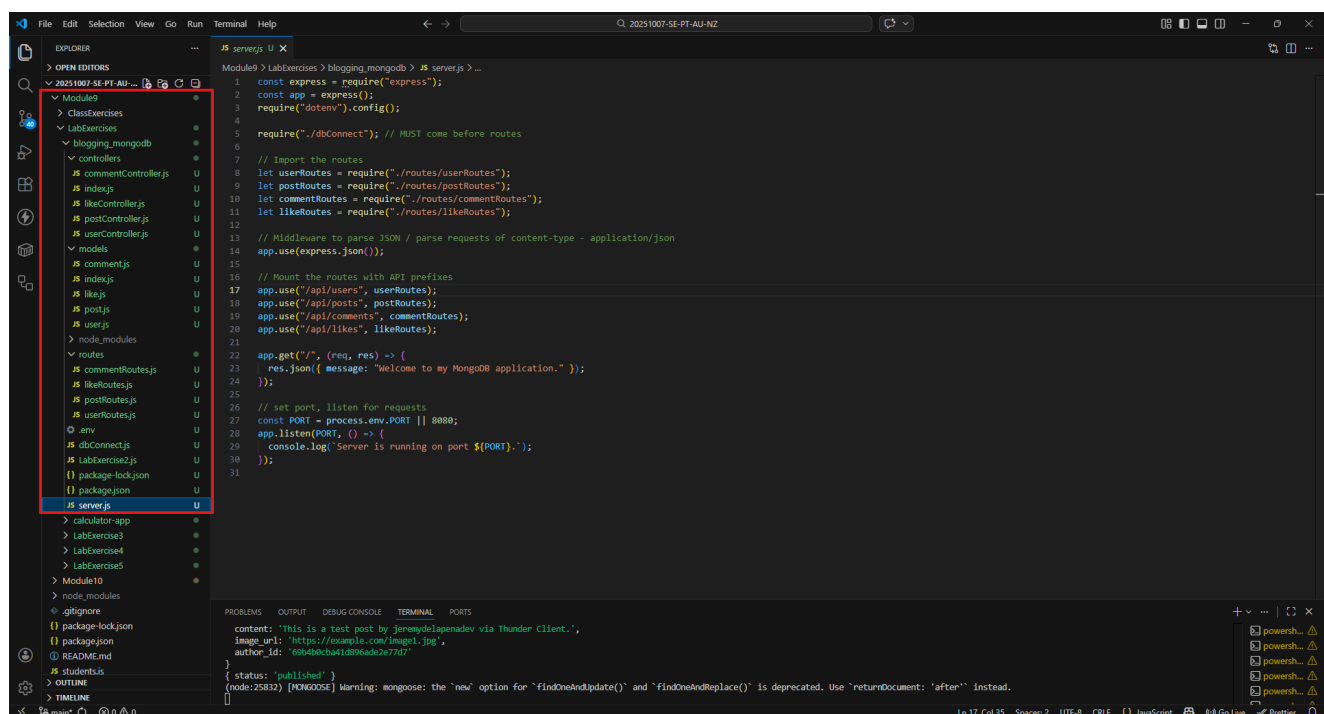
https://drive.google.com/drive/folders/1Az2LS7OiuGDsx1MB7rPtXtk_ak_t6_6W

Create an express back-end application for a Blog website using MongoDB. You should refer to your database model from Module 8 for this, ensuring that your app matches the model.

Requirements:

- Your system should have a proper MVC Structure
- The system should be able to create users.
- The users should be able to create multiple posts (posts should be very basic with title, description and image)
- Other users should be able to like the posts and comment on the posts.

[/] Your system should have a proper MVC Structure



[/] The system should be able to create users.

1.) Creating a user.

The screenshot shows the VS Code interface with a REST client request to `http://localhost:8080/api/users`. The response is a JSON array of two users:

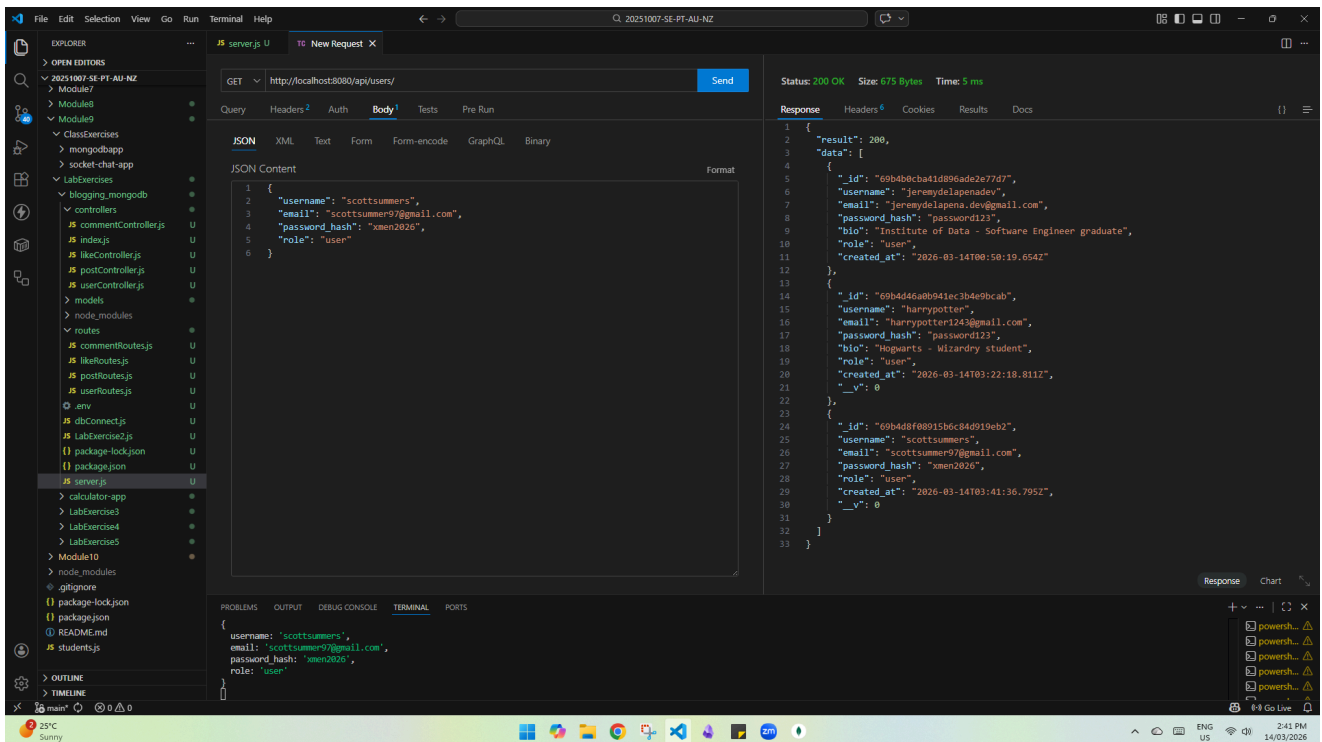
```
{
  "result": 200,
  "data": [
    {
      "_id": "69b4b0c8a41d89eade2e77d7",
      "username": "jerrydelaapena.dev",
      "email": "jerrydelaapena.dev@gmail.com",
      "password_hash": "password123",
      "bio": "Institute of Data - Software Engineer graduate",
      "role": "user",
      "created_at": "2026-03-14T00:50:19.654Z"
    },
    {
      "_id": "69b4d46a0b941ec3b4e9cab",
      "username": "harrypotter",
      "email": "harrypotter123@gmail.com",
      "password_hash": "password123",
      "bio": "Hogwarts - Wizardry student",
      "role": "user",
      "created_at": "2026-03-14T03:22:18.811Z",
      "_v": 0
    }
  ]
}
```

The terminal shows the command `node server.js` and the output `[dotenv@17.3.1] injecting env (2) from .env -- tip: prevent building .env in docker: https://dotenvx.com/prebuild`.

The screenshot shows the VS Code interface with a REST client request to `http://localhost:8080/api/users/create`. The response is a JSON object representing a single user:

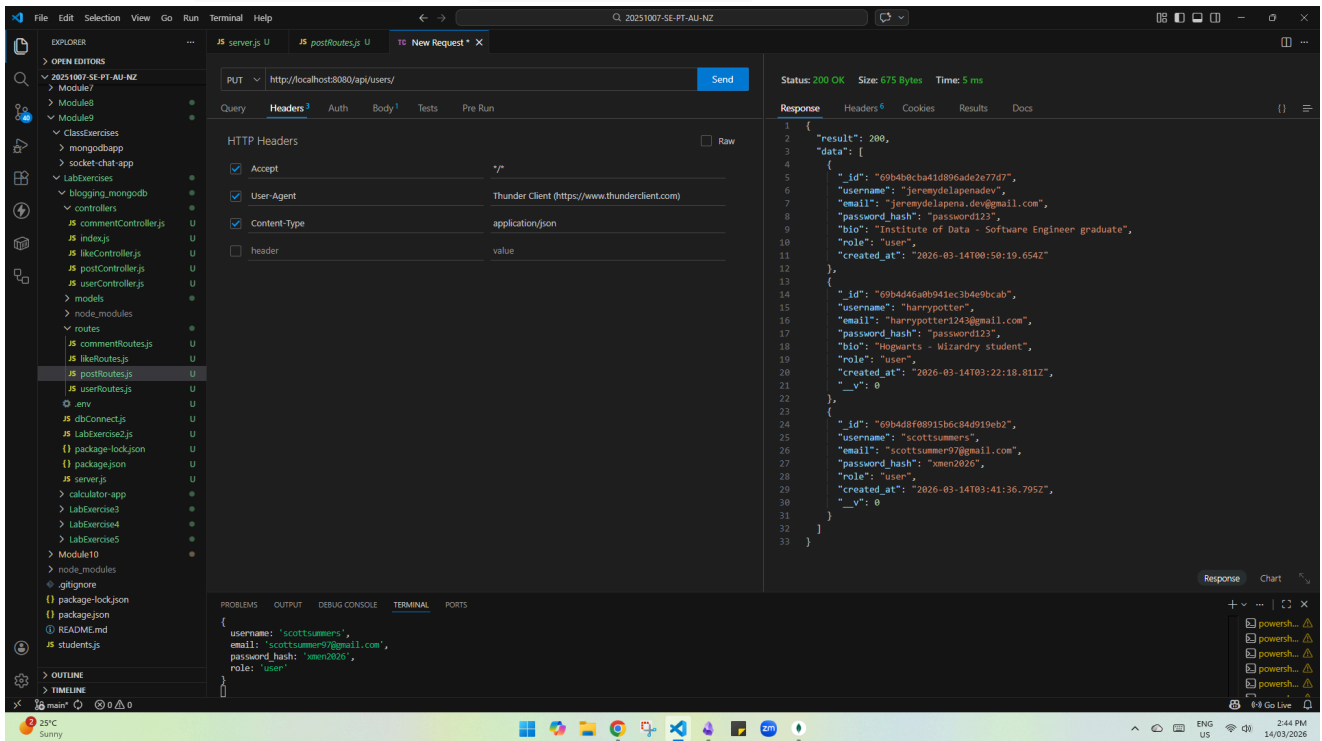
```
{
  "result": 200,
  "data": {
    "username": "scottsummers",
    "email": "scottsummer9@gmail.com",
    "password_hash": "xmen2026",
    "role": "user",
    "_id": "69b4d8f80915b6c84d919eb2",
    "created_at": "2026-03-14T03:41:36.795Z",
    "_v": 0
  }
}
```

The terminal shows the command `node server.js` and the output `{ username: 'scottsummers', email: 'scottsummer9@gmail.com', password_hash: 'xmen2026', role: 'user' }`.



2.) Updating a user.

Note: Added Content-Type and application/json before putting data.



Visual Studio Code interface showing a REST client request and response.

Request:

- Method: PUT
- URL: `http://localhost:8080/api/users/69b4d46ab941ec3b4e9bcab`
- Body (JSON):

```
{  "username": "harrypotter2",  "email": "harryjamespotter@gmail.com"}
```

Response:

- Status: 200 OK
- Size: 675 Bytes
- Time: 4 ms
- Body (JSON):

```
{  "result": 200,  "data": [    {      "_id": "69b4d46ab941ec3b4e9bcab",      "username": "harrypotter2",      "email": "harryjamespotter@gmail.com",      "password_hash": "password123",      "bio": "Institute of Data - Software Engineer graduate",      "role": "user",      "created_at": "2026-03-14T08:58:19.654Z"    },    {      "_id": "69b4d46ab941ec3b4e9bcab",      "username": "harrypotter",      "email": "harrypotter123@gmail.com",      "password_hash": "password123",      "bio": "Hogwarts - Wizardry student",      "role": "user",      "created_at": "2026-03-14T03:22:18.811Z",      "_v": 0    },    {      "_id": "69b4d8f80915b6c84d919eb2",      "username": "scottsummers",      "email": "scottsummer9@gmail.com",      "password_hash": "xmen2026",      "role": "user",      "created_at": "2026-03-14T03:41:36.795Z",      "_v": 0    }  ]}
```

Visual Studio Code interface showing a REST client request and response.

Request:

- Method: PUT
- URL: `http://localhost:8080/api/users/69b4d46ab941ec3b4e9bcab`
- Body (JSON):

```
{  "username": "harrypotter2",  "email": "harryjamespotter@gmail.com"}
```

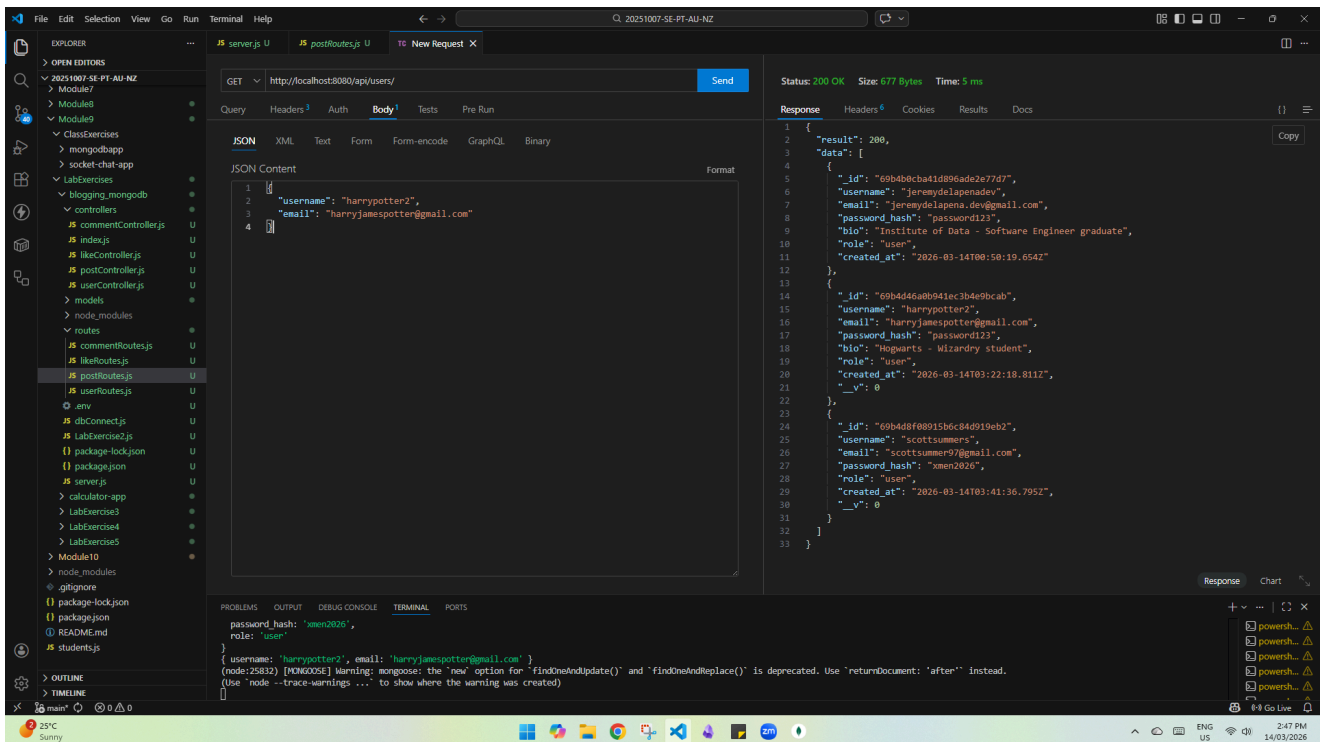
Response:

- Status: 200 OK
- Size: 247 Bytes
- Time: 17 ms
- Body (JSON):

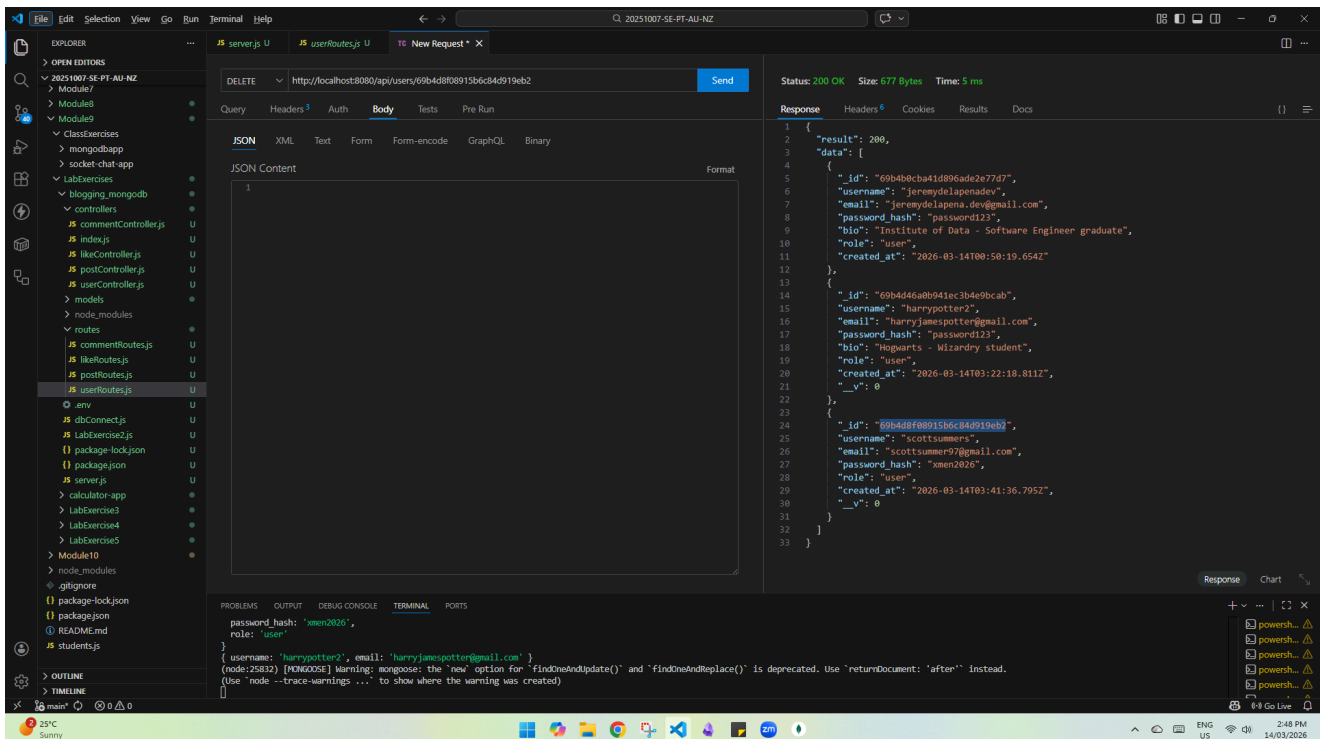
```
{  "result": 200,  "data": {    "_id": "69b4d46ab941ec3b4e9bcab",    "username": "harrypotter2",    "email": "harryjamespotter@gmail.com",    "password_hash": "password123",    "bio": "Hogwarts - Wizardry student",    "role": "user",    "created_at": "2026-03-14T03:22:18.811Z",    "_v": 0  }}
```

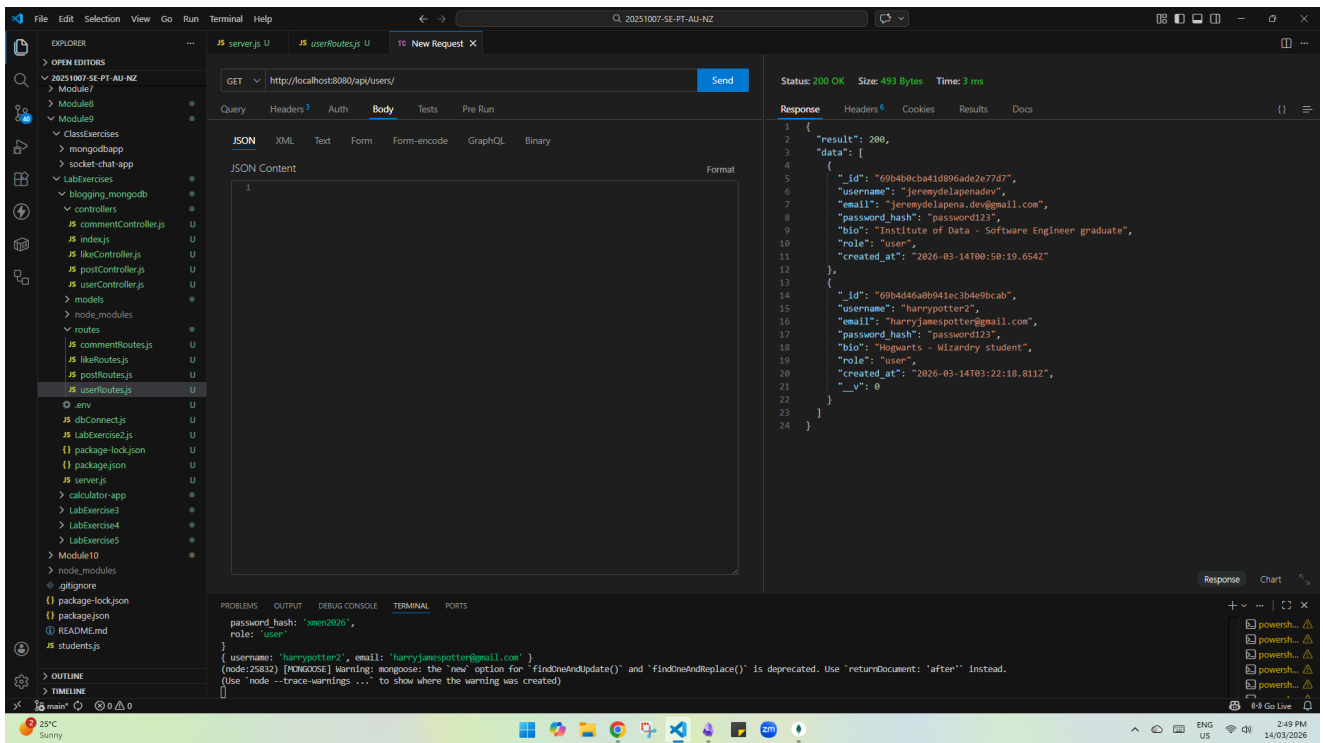
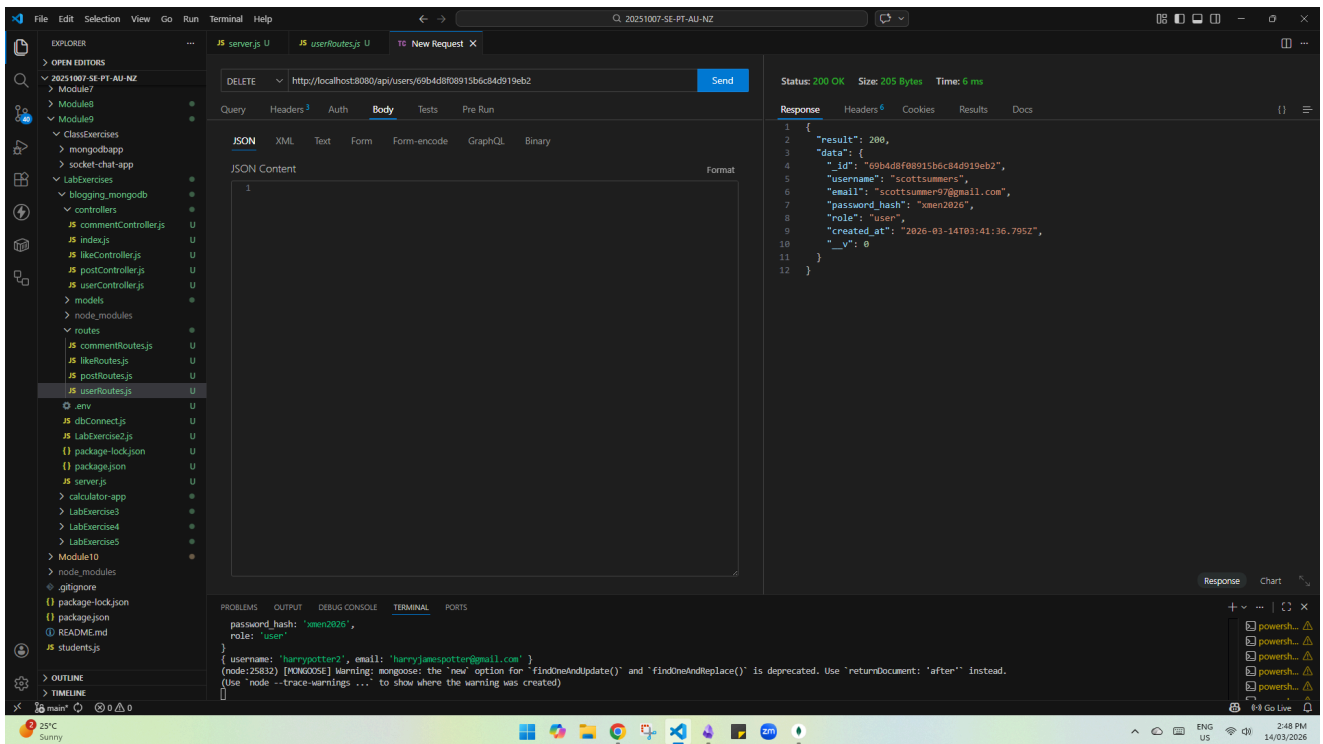
Terminal Output:

```
password_hash: 'xmen2026',  
role: 'user'  
{ username: 'harrypotter2', email: 'harryjamespotter@gmail.com' }  
(node:25832) [WARNING] mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.  
(Use 'node --trace-warnings ...' to show where the warning was created)
```



3.) Deleting a user





[/] The users should be able to create multiple posts (posts should be very basic with title, description and image)

VS Code interface showing a REST client request and response. The request is a POST to `http://localhost:8080/api/posts/create` with a JSON body. The response is a 200 OK status with a JSON body.

```
POST http://localhost:8080/api/posts/create

{
  "title": "My Second Blog Post",
  "content": "This is a test post by jeremydelapenadev via Thunder Client.",
  "image_url": "https://example.com/image1.jpg",
  "author_id": "69b4bcb41d896ade2e77d7"
}
```

```
{
  "result": 200,
  "data": [
    {
      "_id": "69b4b17ba41d896ade2e77d8",
      "title": "My First Blog Post",
      "content": "This information has been updated using the Update operation.",
      "status": "published",
      "image_url": "https://example.com/blogImage.jpg",
      "author_id": "69b4bcb41d896ade2e77d7",
      "created_at": "2026-03-14T00:53:15.546Z",
      "updated_at": "2026-03-14T00:53:15.546Z"
    }
  ]
}
```

VS Code interface showing a REST client request and response. The request is a POST to `http://localhost:8080/api/posts/create` with a JSON body. The response is a 200 OK status with a JSON body.

```
POST http://localhost:8080/api/posts/create

{
  "title": "My Second Blog Post",
  "content": "This is a test post by jeremydelapenadev via Thunder Client.",
  "image_url": "https://example.com/image1.jpg",
  "author_id": "69b4bcb41d896ade2e77d7"
}
```

```
{
  "result": 200,
  "data": {
    "title": "My Second Blog Post",
    "content": "This is a test post by jeremydelapenadev via Thunder Client.",
    "status": "draft",
    "image_url": "https://example.com/image1.jpg",
    "author_id": "69b4bcb41d896ade2e77d7",
    "_id": "69b4db688915b684d919ebb",
    "created_at": "2026-03-14T03:52:08.528Z",
    "updated_at": "2026-03-14T03:52:08.528Z",
    "_v": 0
  }
}
```

Additional: Updating the status from draft to published

The screenshot shows the VS Code interface with a REST client request in the 'Body' tab. The request is a PUT to `http://localhost:8080/api/posts/69b4db688915b6c84d919ebb`. The JSON body is:

```
{
  "status": "published"
}
```

The response is shown in the 'Response' tab, indicating a 200 OK status. The response body is:

```
{
  "result": 200,
  "data": {
    "_id": "69b4b17ba41d896ade2e77d8",
    "title": "My First Blog Post",
    "content": "This information has been updated using the Update operation.",
    "status": "published",
    "image_url": "https://example.com/blogImage.jpg",
    "author_id": "69b4db688915b6c84d919ebb",
    "created_at": "2026-03-14T00:53:15.546Z",
    "updated_at": "2026-03-14T00:53:15.546Z"
  }
}
```

The screenshot shows the VS Code interface with a REST client request in the 'Body' tab. The request is a PUT to `http://localhost:8080/api/posts/69b4db688915b6c84d919ebb`. The JSON body is:

```
{
  "status": "published"
}
```

The response is shown in the 'Response' tab, indicating a 200 OK status. The response body is:

```
{
  "result": 200,
  "data": {
    "_id": "69b4db688915b6c84d919ebb",
    "title": "My Second Blog Post",
    "content": "This is a test post by jeremydelapenadev via Thunder Client.",
    "status": "published",
    "image_url": "https://example.com/image1.jpg",
    "author_id": "69b4b0cba41d896ade2e77d7",
    "created_at": "2026-03-14T03:52:08.528Z",
    "updated_at": "2026-03-14T03:52:22.337Z",
    "__v": 0
  }
}
```

A warning message is visible in the terminal:

```
(node:25832) [MONGODB] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
```

[/] Other users should be able to like the posts.

user_id of jeremydelapenadev : 69b4b0cba41d896ade2e77d7

user_id of harrypotter2 : 69b4d46a0b941ec3b4e9bcab

post_id of jeremydelapenadev : 69b4b17ba41d896ade2e77d8 (First Blog Post)

post_id of jeremydelapenadev : 69b4db688915b6c84d919ebb (Second Blog Post)

VS Code interface showing a REST client request and response. The request is a POST to `http://localhost:8080/api/likes/create` with a JSON body:

```
1 {
2   "user_id": "69b4d46a8b941ec3b4e9bcab",
3   "post_id": "69b4dbcb41d896ade2e77d7"
4 }
```

The response is a 200 OK status with a JSON body:

```
1 {
2   "result": 200,
3   "data": [
4     {
5       "_id": "69b4b17ba41d896ade2e77d8",
6       "title": "My First Blog Post",
7       "content": "This information has been updated using the Update operation.",
8       "status": "published",
9       "image_url": "https://example.com/blogImage.jpg",
10      "author_id": "69b4dbcb41d896ade2e77d7",
11      "created_at": "2026-03-14T00:53:15.546Z",
12      "updated_at": "2026-03-14T00:53:15.546Z"
13    },
14    {
15      "_id": "69b4db88915b6c84d919ebb",
16      "title": "My Second Blog Post",
17      "content": "This is a test post by jeremydelapenadv via Thunder Client.",
18      "status": "published",
19      "image_url": "https://example.com/image1.jpg",
20      "author_id": "69b4dbcb41d896ade2e77d7",
21      "created_at": "2026-03-14T03:52:00.528Z",
22      "updated_at": "2026-03-14T03:52:22.337Z",
23      "_v": 0
24    }
25  ]
26 }
```

The PROBLEMS panel shows an error:

```
reason: undefined,
Symbol(MongooseValidationError): true
_message: 'like validation failed'
```

VS Code interface showing a REST client request and response. The request is a POST to `http://localhost:8080/api/likes/create` with a JSON body:

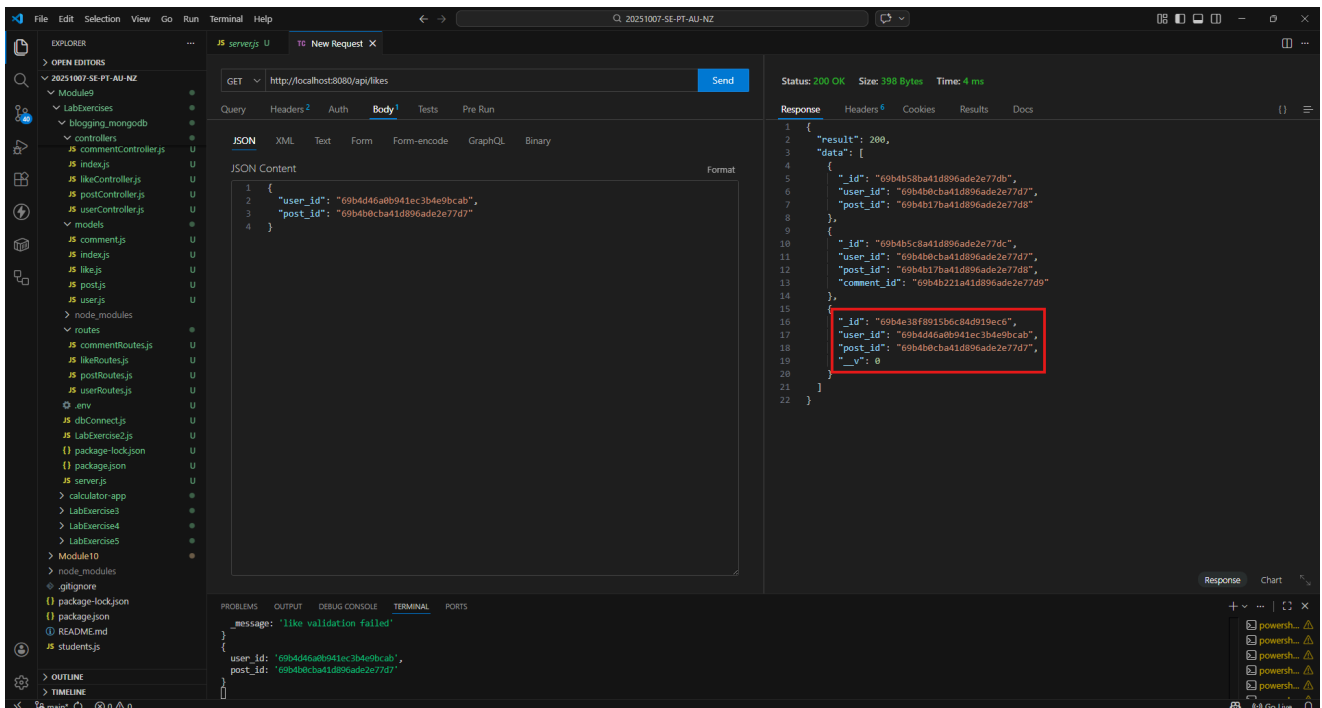
```
1 {
2   "user_id": "69b4d46a8b941ec3b4e9bcab",
3   "post_id": "69b4dbcb41d896ade2e77d7"
4 }
```

The response is a 200 OK status with a JSON body:

```
1 {
2   "result": 200,
3   "data": {
4     "user_id": "69b4d46a8b941ec3b4e9bcab",
5     "post_id": "69b4dbcb41d896ade2e77d7",
6     "_id": "69b4e38f8915b6c84d919ec6",
7     "_v": 0
8   }
9 }
```

The PROBLEMS panel shows an error:

```
_message: 'like validation failed'
{
  user_id: '69b4d46a8b941ec3b4e9bcab',
  post_id: '69b4dbcb41d896ade2e77d7'
}
```



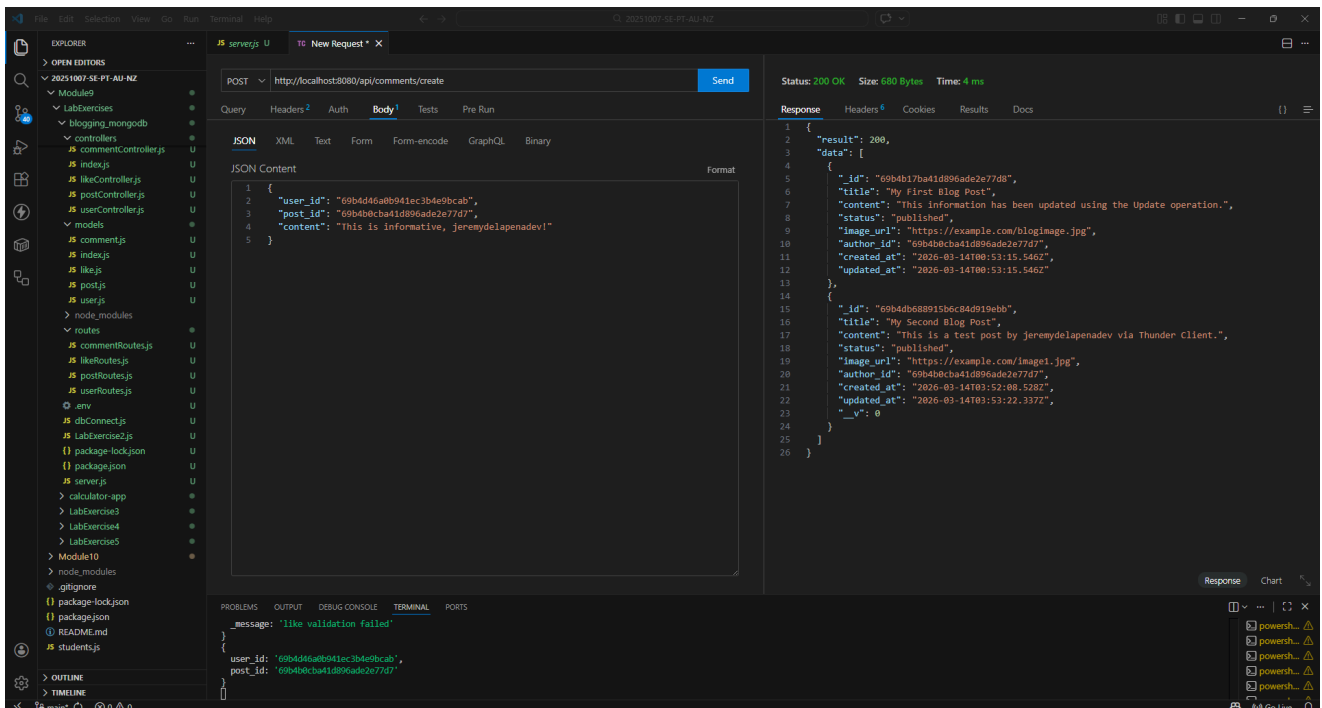
[/] Other users should be able to comment on the posts.

user_id of jeremydelapenadev : 69b4b0cba41d896ade2e77d7

user_id of harrypotter2 : 69b4d46a0b941ec3b4e9bcab

post_id of jeremydelapenadev : 69b4b17ba41d896ade2e77d8 (First Blog Post)

post_id of jeremydelapenadev : 69b4db688915b6c84d919ebb (Second Blog Post)



VS Code interface showing a REST client request and response.

Request:

- Method: POST
- URL: http://localhost:8080/api/comments/create
- Body (JSON):

```
{  "user_id": "69b4d46a8b941ec3b4e9bcab",  "post_id": "69b4bcb41d896ade2e77d7",  "content": "This is informative, jeremydelapenadev!"}
```

Response:

- Status: 200 OK
- Size: 230 Bytes
- Time: 5 ms
- Body (JSON):

```
{  "result": 200,  "data": {    "content": "This is informative, jeremydelapenadev!",    "post_id": "69b4bcb41d896ade2e77d7",    "user_id": "69b4d46a8b941ec3b4e9bcab",    "id": "69b4e7178915b6c84d919eca",    "created_at": "2026-03-14T04:41:59.206Z",    "_v": 0  }  }
```

Terminal:

```
{  "user_id": "69b4d46a8b941ec3b4e9bcab",  "post_id": "69b4bcb41d896ade2e77d7",  "content": "This is informative, jeremydelapenadev!"}
```

VS Code interface showing a REST client request and response.

Request:

- Method: GET
- URL: http://localhost:8080/api/comments/
- Body (JSON):

```
{  "user_id": "69b4d46a8b941ec3b4e9bcab",  "post_id": "69b4bcb41d896ade2e77d7",  "content": "This is informative, jeremydelapenadev!"}
```

Response:

- Status: 200 OK
- Size: 445 Bytes
- Time: 4 ms
- Body (JSON):

```
{  "result": 200,  "data": [    {      "id": "69b4b221a1d896ade2e77d0",      "content": "Congrats for successfully updating your first post!",      "post_id": "69b4b17ba1d896ade2e77d8",      "user_id": "69b4bcb41d896ade2e77d7",      "created_at": "2026-03-14T00:56:00.998Z"    },    {      "id": "69b4e7178915b6c84d919eca",      "content": "This is informative, jeremydelapenadev!",      "post_id": "69b4bcb41d896ade2e77d7",      "user_id": "69b4d46a8b941ec3b4e9bcab",      "created_at": "2026-03-14T04:41:59.206Z",      "_v": 0    }  ]}
```

Terminal:

```
{  "user_id": "69b4d46a8b941ec3b4e9bcab",  "post_id": "69b4bcb41d896ade2e77d7",  "content": "This is informative, jeremydelapenadev!"}
```